Spring Security

Part 4 — BCrypt Theory, User Registration and Password Encoding

1. What is BCrypt

Till this point we were able to store users in the database and validate them using username and password with Spring Security. But there is a problem. If we look at the database we are storing passwords in plain text. Moreover we are also transferring this data in plain text. This is a serious security risk and we need to fix it.

2. The Problem with Plain Text Passwords

When we store passwords in a database or transfer them over networks in plain text, anyone who gains access to the database or intercepts the network transfer can immediately see all passwords. So we need some way to secure these passwords before storing or transferring them. The first solution that comes to mind is cryptography.

3. Encryption (Two Way)

Cryptography is a concept where we encrypt a message and then on the other end we can decrypt it. Think of it like a lock and key. When storing a password we encrypt it with a key and when we want to verify the password we decrypt it with the same key and then verify it.

This sounds good but there is a fundamental problem. The key has to be stored somewhere on the server. If that key is ever leaked all passwords are leaked at once because anyone with the key can decrypt everything. So while encryption protects the passwords it creates a new single point of failure which is the key itself. We need a better approach.

4. Hashing (One Way)

The better approach is one way encryption which is called hashing. The key difference from encryption is that hashing is one way. Once you hash something you cannot reverse it to get the original value back. So instead of storing an encrypted password that can be decrypted, we store a hash that can never be reversed.

Hashing converts data into a fixed length string of characters. Every unique input will always produce the same unique hash. There is no collision. To verify a password we do not decrypt anything. Every time a user enters their password we convert it to a hash again and compare the two hashes. If they match the password is correct. The original password never needs to be stored or decrypted anywhere.

Popular hashing algorithms are MD5 and SHA256. But these are general purpose fast hashing algorithms. Fast sounds good but for passwords fast is actually a problem because an attacker can try billions of combinations per second using powerful hardware. Also these algorithms do not add salt by default, meaning if two users have the same password they will produce the same hash. This makes it easier to crack using precomputed tables called rainbow tables. So we need something better than plain SHA or MD5 for passwords.

5. What is Salt

Before we get to BCrypt we need to understand what salt is because it is one of the key reasons BCrypt is better. Salt is a random value that is automatically generated and added to your password before hashing it. Because the salt is random and unique every time, even if two users have the exact same password their final hash will look completely different. This completely eliminates the rainbow table attack because every hash is unique even for identical passwords. BCrypt handles salt automatically so you do not need to manage it yourself.

6. The Secure Solution — BCrypt

Now that we understand the problems with plain text, encryption, and basic hashing, we can understand why BCrypt is the right solution for passwords. BCrypt is a specialized password hashing function designed specifically for security. It solves all the problems we discussed above.

BCrypt is completely different from SHA256 and MD5. SHA256 and MD5 were designed for speed with no built in salt. BCrypt is password specific, intentionally slow, and has salt built in automatically. The way BCrypt achieves extra security on top of hashing is by doing it in multiple rounds. Basic algorithms like SHA256 convert your text into a hash only once. BCrypt repeats this process many times. For example with a cost factor of 10 it does 2 to the power of 10 which is 1024 rounds of hashing. This makes it extremely difficult for an attacker to crack because they have to repeat the same number of rounds for every single guess they make.

7. How BCrypt Works

With this understanding the login and registration flow with BCrypt becomes simple. When a user registers, BCrypt hashes their password and that hash is stored in the database. The original password is never stored anywhere. When the user tries to log in, BCrypt hashes the password they entered and the system compares this new hash with the stored hash. If they match the password is correct and login succeeds. If not it fails. The original password is never recovered or compared directly at any point.

8. BCrypt Example

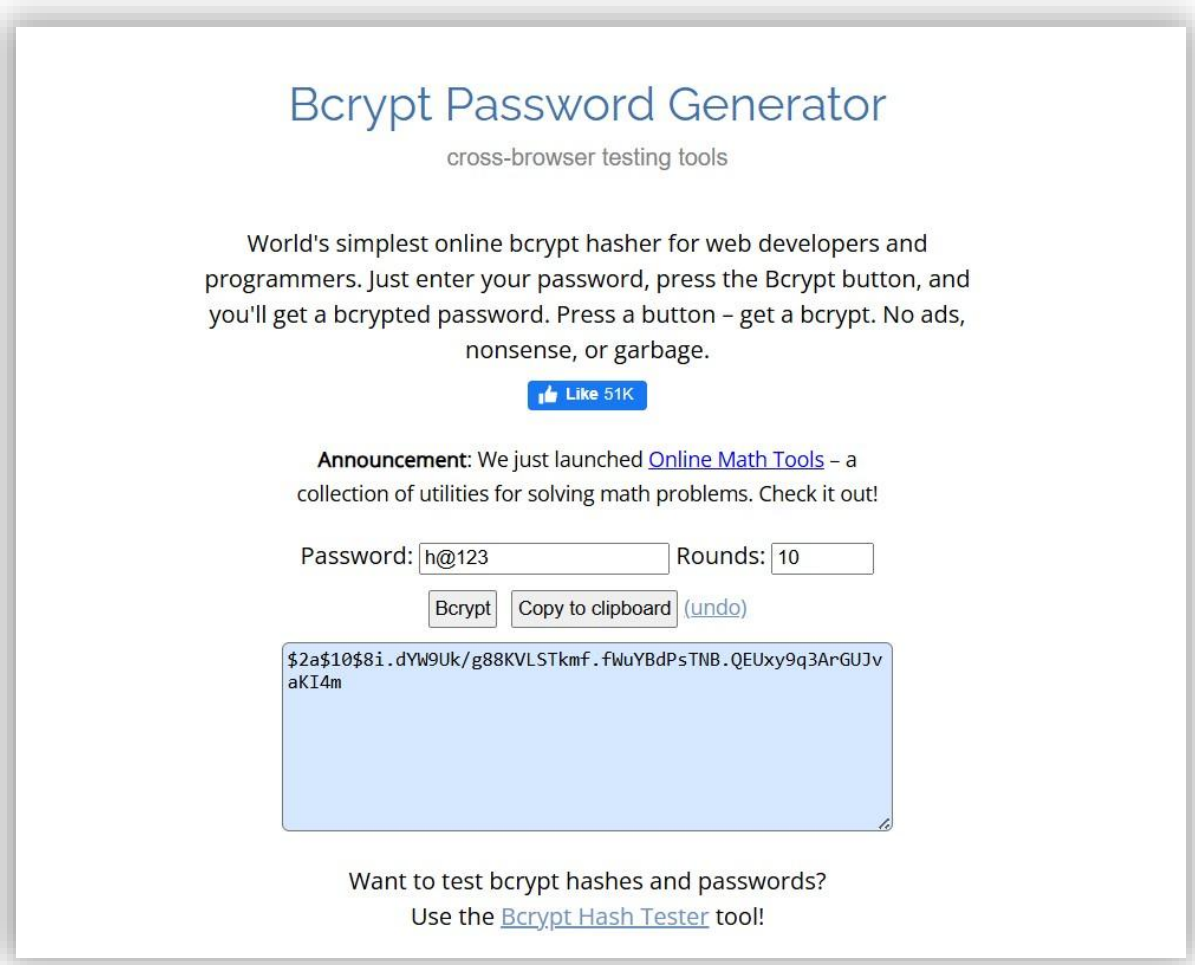
You can see BCrypt in action at https://www.bcrypt-generator.com

For password H@123 BCrypt will produce something like this.

```
$2a$10$iXuLkFOjGmdSpn4W5A5n9uEXk5TEF0DD8uuwjWpvUXqBuH8uXuBC.
```

$2a indicates the BCrypt version. $10 indicates the cost factor meaning 2 to the power of 10 rounds which is 1024 iterations, not just 10. The rest of the string is the salt and the hashed password combined together. The higher the cost factor the more secure but slightly slower the processing becomes. This cost factor can be adjusted over time as hardware gets faster to always stay ahead of attackers.

You can see Bcrypt in action at https://www.browsrling.com



9. Key Features of BCrypt

Slow by design to prevent brute force attacks.

Built in salt protection against rainbow table attacks.

Adjustable work factor to adapt to faster computers over time.

Industry standard for password security.

10. SHA vs BCrypt — Simple Comparison

SHA256 and MD5	BCrypt
General purpose hashing	Password specific hashing

Designed for speed No built in salt	Designed to be slow intentionally Built in salt automatically
Vulnerable to rainbow table attacks Single round of hashing Not suitable for passwords	Protected against rainbow table attacks Multiple rounds of hashing Industry standard for passwords

11. When You Join a Job as a Backend Developer

Now that you understand how passwords should be stored securely, here is where this fits in a real job. When you join a company as a backend developer the project structure will usually already exist. But whether you are building from scratch or adding a feature the hierarchy always follows this pattern.

config — SecurityConfig and other configuration classes.

model — entity classes like User that map to database tables.

dao — repository interfaces extending JpaRepository.

service — service classes containing business logic.

controllers — REST controllers handling incoming HTTP requests.

The flow always goes like this. Request comes in to Controller. Controller calls Service. Service calls Repository. Repository talks to Database. Response travels back up the same chain.

When you get a task like add user registration you will create or modify the User entity, write a save method in the repo, write registration logic in the service, and expose it through a controller endpoint. In that registration logic you will use BCrypt to hash the password before saving it to the database. SecurityConfig will define which endpoints are open and which require authentication. That is your complete job as a backend developer and now you understand every piece of it.

> *BCrypt is used only at registration to hash the password before saving. At login you never hash and compare manually. Spring Security and BCryptPasswordEncoder handle the comparison automatically when you set it as the password encoder in your SecurityConfig.*

Next up — BCrypt practical implementation, User Registration endpoint, and

Practical — User Registration and BCrypt Implementation

12. User Registration

Now we have to implement the password encoder which is BCrypt. But before that we also want to create a registration option so that a user can register themselves.

Create a controller in the controllers package named UserController. Mark it with @RestController and create a POST endpoint for registration. We will keep this endpoint public because a user who is registering does not have credentials yet. This method will accept a User object and return it.

```
@RestController
public class UserController {

    @PostMapping("register")
    public User register(@RequestBody User user) {
        return user;
    }
}
```

We also want to store the user in the database. To do that we create a service class and add a method to save the user. From the controller we pass the user object to the service so it can save it in the database. The service needs the repo layer for that. We already have UserRepo so we inject it using @Autowired and use the pre-defined save method that JPA provides.

```
@Service
public class UserService {

    @Autowired
    private UserRepo repo;

    public User saveUser(User user) {
        return repo.save(user);
    }
}
```

Now inject UserService in UserController and call saveUser from the register method.

```
@RestController
public class UserController {

    @Autowired
    private UserService service;

    @PostMapping("register")
    public User register(@RequestBody User user) {
        return service.saveUser(user);
    }
}
```

Test this in Postman by hitting the register endpoint with a POST request. Add the user data in the request body in JSON format. In the Authorization tab use any username and password that already exists in the database. The user will be created successfully and stored in the database.

13. BCrypt Encoding for User Registration

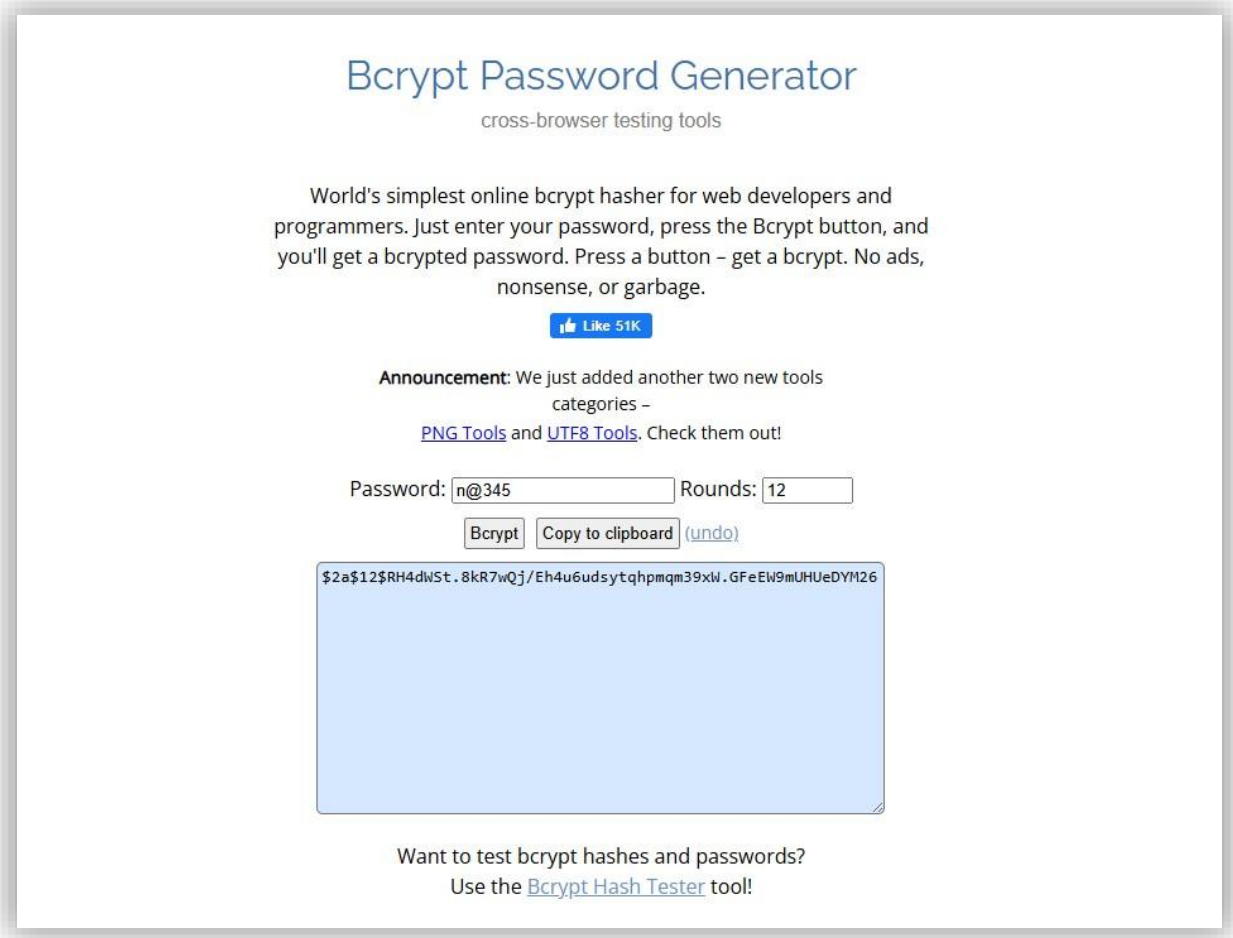
Now let us implement BCrypt. Currently even though we have set up registration, the password is still being stored in plain text in the database. We need to encode it using BCrypt before saving.

The Challenge With Existing Data

When we implement BCrypt in an existing application we face a challenge. Existing users already have plain text passwords in the database. Our authentication system currently expects plain text passwords. Once BCrypt is implemented the system will try to compare BCrypt hashes with plain text passwords and authentication will fail for all existing users.

This is why before implementing BCrypt we need to update the existing passwords in the database first. We convert the plain text passwords to BCrypt encoded versions using an online tool like https://www.bcrypt-generator.com and then run an update query in the database to replace the plain text passwords with the encoded ones. This way all existing users will have BCrypt hashes stored before we switch the system to expect BCrypt.

You can generate encoded passwords using an online tool like https://www.browsrling.com:



Query Query History

```
SELECT * FROM users;

update users set password='$2a$12$RH4dWSt.8kR7wQj/Eh4u6udsytqhpmqm39xW.GFeEW9mUHUeDYM26' where id=3
```

Data Output Messages Notifications

	id [PK] integer	username text	password text
1	1	kiran	n@789
2	2	harsh	h@123
3	3	navin	$2a$12$RH4dWSt.8kR7wQj/Eh4u6udsytqhpmqm39xW.GFeEW9mUHUeDYM26

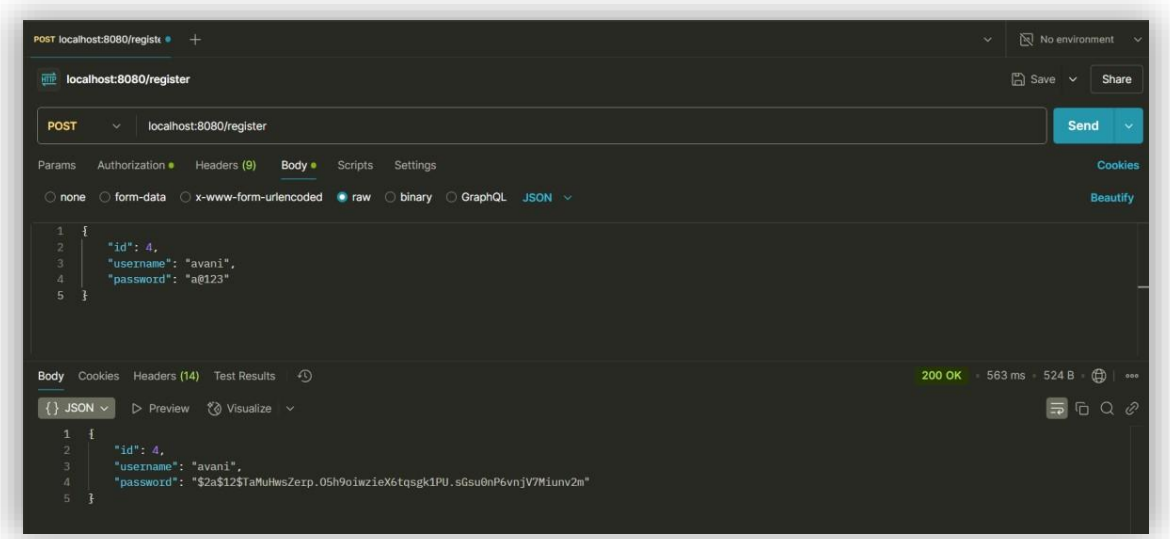
The reason we cannot just log in with a plain text password after implementing BCrypt is that BCrypt hashing is one way. When you log in, the system takes your entered password, runs it through BCrypt, and compares the resulting hash with whatever is stored. If the stored password is plain text and the entered password produces a BCrypt hash, they will never match. So all passwords in the database must be in BCrypt format before you enable BCrypt authentication.

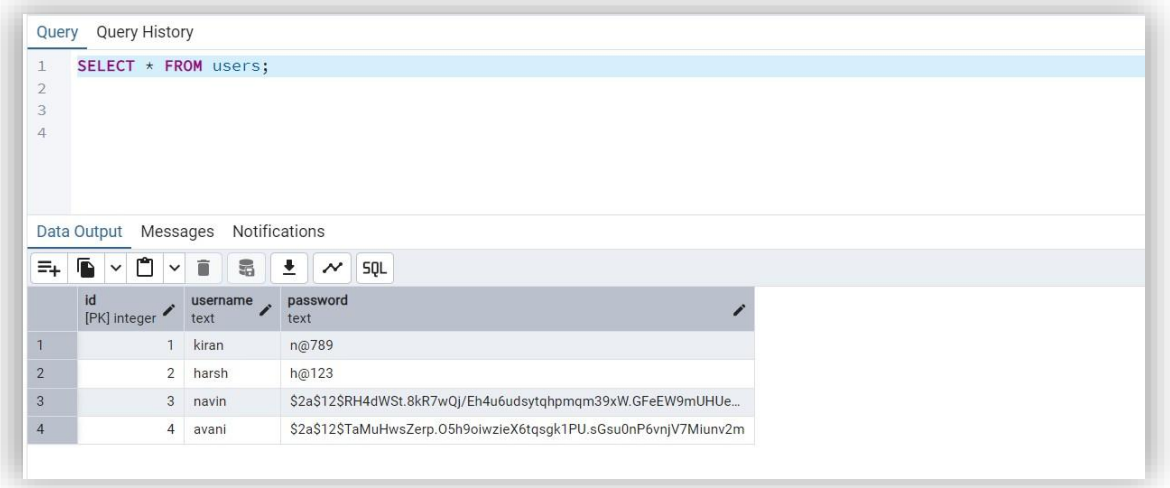
Implementing BCrypt in the Service Layer

In the UserService class we create a BCryptPasswordEncoder object directly using new. BCryptPasswordEncoder is part of the Spring Security framework so we do not need to add any extra library. We set the strength parameter to 12 which means 2 to the power of 12 which is 4096 rounds of hashing. Before saving the user we encode the password using this encoder and then save the user with the encoded password to the database.

```
@Service
public class UserService {

    @Autowired
    private UserRepo repo;

    private BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);

    public User saveUser(User user){
        user.setPassword(encoder.encode((user.getPassword())));
        return repo.save(user);
    }
}
```

Output:





Things to Remember

BCryptPasswordEncoder is part of Spring Security so no extra library is needed.

The parameter 12 represents the work factor meaning 2 to the power of 12 which is 4096 rounds.

Higher work factor means better security but slightly slower performance.

The encode() method transforms the plain text password into a BCrypt hash.

After this implementation all new users will have encoded passwords stored in the database.

> *Never print encoded passwords to the console in production. Only do this for demonstration during learning.*

14. Setting Password Encoder in SecurityConfig

Till now we have encoded the password when a user registers so it gets stored in encoded format in the database. Now let us handle authentication, meaning when the user logs in again how do we authenticate the entered password against the stored encoded password.

Previously we were using NoOpPasswordEncoder in SecurityConfig which expects plain text passwords and compares them directly. Now that our passwords are stored as BCrypt hashes we need to replace NoOpPasswordEncoder with BCryptPasswordEncoder in SecurityConfig. This tells Spring Security to use BCrypt when verifying passwords during login.

After setting BCryptPasswordEncoder as the encoder in SecurityConfig, if you try to log in with a user whose password is still stored as plain text in the database it will fail. This is because Spring Security will now take the entered password, run it through BCrypt, and try to match the resulting hash with whatever is stored. Plain text will never match a BCrypt hash. So only users who have BCrypt encoded passwords stored in the database can log in successfully. This is exactly why we had to update the existing passwords in the database before making this change.

In SecurityConfig replace NoOpPasswordEncoder with BCryptPasswordEncoder using the same strength parameter of 12 that we used during registration.

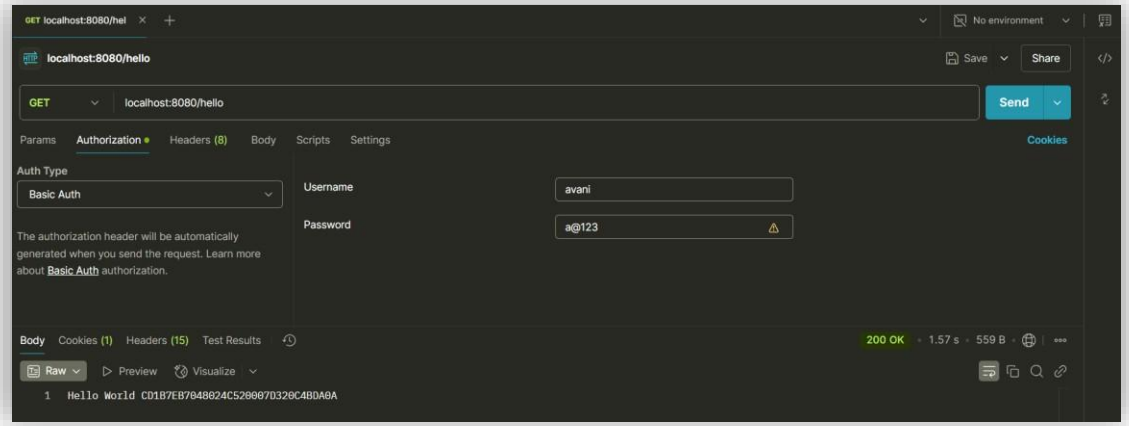
```
@Bean
public AuthenticationProvider authProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService);
    provider.setPasswordEncoder(new BCryptPasswordEncoder(12));
    return provider;
}
```

Or to see full code and output refer below ->

Example:

```java
package com.telusko.springsecdemo.config;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Autowired
    private UserDetailsService userDetailsService;

    @Bean
    public AuthenticationProvider authProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        provider.setPasswordEncoder(new BCryptPasswordEncoder(12));
        return provider;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.csrf(customizer -> customizer.disable())
                .authorizeHttpRequests(request -> request.anyRequest().authenticated())
                .httpBasic(Customizer.withDefaults())
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        return http.build();
    }
}
```

Output:



Complete Authentication Flow With BCrypt

User provides username and password. Spring Security retrieves the user details from the database using UserDetailsService. BCryptPasswordEncoder takes the plain text password from the login attempt and applies the same BCrypt hashing algorithm with the same cost factor. It then compares this newly generated hash with the stored hash from the database. If they match authentication succeeds. If not it fails.

> *The cost factor used in BCryptPasswordEncoder during registration and during authentication must always be the same. If you register with 12 rounds you must also verify with 12 rounds.*

> ***Next up — JWT (JSON Web Token) and OAuth2.***